LAB 7 REPORT: FUZZ TESTING & INPUT VALIDATION

Dynamic Security Testing, Property-Based Fuzzing with Hypothesis, and Defensive Programming

Student Name: Aniket Sen	**SRN:** PES1UG24CS062	**Class & Section:** 5th Sem, Sec 5A
Course: Software Testing & QA	**Repository:** L6_L7_56_course_registration	**Module:** LAB7 (Course Parser)

1. Assignment Title & Objective

Title: University Lab 7 — Fuzzing for Input Validation Bugs (Course Registration Parser)
Objective: To explore fuzz testing as an automated dynamic security and quality assurance methodology. The project analyzes an existing Python module (course_registration_parser.py) that makes naive assumptions about input formatting. By implementing property-based fuzz tests with the **Hypothesis** library, we systematically trigger unhandled exceptions and uncover edge cases. Subsequently, the application is refactored using defensive programming and strict input validation to ensure all functions handle arbitrary, unexpected, or adversarial inputs safely without crashing.

2. Fuzz Testing Methodology & Security Importance

What is Fuzz Testing? Fuzz testing (or *fuzzing*) is a dynamic software testing technique where random, unexpected, or malformed data is automatically generated and supplied to an application's entry points. The primary goals are discovering unhandled crashes, detecting memory or resource leaks, identifying security vulnerabilities, and validating system robustness against edge cases.

Why Use Fuzzing? Manual tests and traditional unit tests inherently test what the engineer *anticipates* could go wrong. Fuzz testing, conversely, discovers edge cases and pathological inputs that developers never imagined. Improper input validation (CWE-20) is among the most prevalent sources of software security vulnerabilities, leading to Denial of Service (DoS) attacks, injection bugs, and state corruption. Fuzzing ensures that software adheres to robust failure semantics — failing safely rather than terminating abruptly.

3. The Hypothesis Property-Based Testing Framework

Hypothesis is a state-of-the-art property-based testing and fuzzing library for Python. Rather than asserting outputs for individual fixed inputs (e.g. assert func('cs 301') == 'CS301'), property-based testing defines fundamental properties or invariants (e.g. *'the function must never raise an unhandled exception for any text or None input'*) and lets Hypothesis synthesize hundreds of pseudo-random inputs to break those properties.

- **Input Strategies:** Strategies like st.text() and st.none() generate random Unicode strings, null characters (\x00), whitespace variants, negative numbers, extreme lengths, and malformed symbols.
- **Automated Shrinking:** When Hypothesis finds a failing input, it applies a test-case reduction algorithm to find the minimal, simplest reproduction case (e.g., shrinking a 200-char string to '' or None).
- **Regression Replay Database:** Discovered failures are persisted locally in .hypothesis/ so that regressions are immediately re-tested on subsequent test invocations.

4. Target Functions in Course Registration Parser

Function	Intended Purpose	Original Implementation Weaknesses
`format_course_code(code)`	Formats raw course codes (e.g. 'cs 301') into standardized uppercase format ('CS301').	Assumes code has exactly 2 space-separated words. Calling code.split() directly crashes on None (AttributeError), empty strings (ValueError), single words (ValueError), and >2 words.
`total_credits(text)`	Calculates the sum of a comma-separated list of course credits (e.g. '4,3,4' -> 11).	Assumes all elements are valid integers. Splitting and casting directly crashes on None (AttributeError), empty string '' (ValueError), missing values '4,,3', and negative/huge credits.
`parse_student_record(line)`	Parses a pipe-delimited record 'SRN|name|status' into a structured dictionary.	Assumes exactly 2 pipes. Directly indexes parts[0], parts[1], parts[2], causing immediate IndexError: list index out of range crashes on '', '|', or missing fields.

5. SS1 — Original Buggy Application Execution

The original course_registration_parser.py was executed with standard valid sample inputs: 'cs 301', '4,3,4', and 'pes1ug23cs001|Asha|regular'. Under ideal happy-path inputs, the application operates normally, masking catastrophic edge-case flaws.



Figure 1: SS1 — Terminal output of original buggy application running on happy-path inputs.

6. SS2 — Fuzz Tests Failing on Original Implementation

The completed Hypothesis fuzz testing suite in test_course_registration_parser.py was executed against the original code using pytest -v. Hypothesis immediately generated boundary inputs that crashed all three functions with unhandled exceptions (100% failure rate).

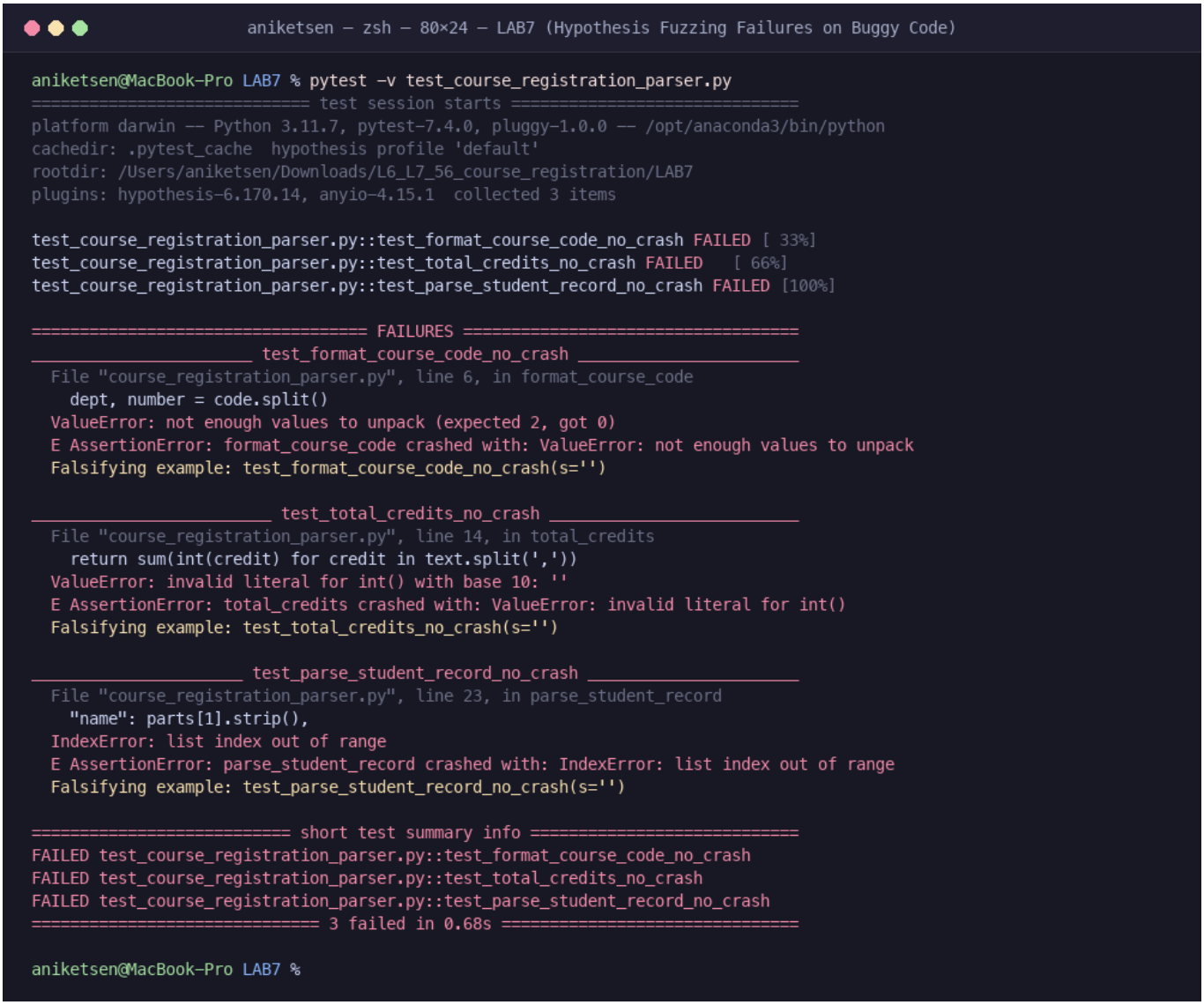


Figure 2: SS2 — Pytest test runner output showing 3 failing fuzz tests with Hypothesis minimized counterexamples.

7. Summary Table of Discovered Vulnerabilities

Function	Falsifying Input	Exception	Root Cause & Vulnerability
`format_course_code`	`s = ''` `s = None`	ValueError AttributeError	Unchecked tuple unpacking dept, number = [ ] and missing isinstance(code, str) check.
`total_credits`	`s = ''` `s = None`	ValueError AttributeError	Splitting empty string yields ['], crashing on int('); missing None validation.
`parse_student_record`	`s = '', s = '|'` `s = None`	IndexError AttributeError	Directly indexing parts[1], parts[2] when fewer than 3 elements exist; missing None validation.

8. Defensive Programming Fixes Implemented

To eliminate all crash surfaces while maintaining a clear and predictable input contract, course_registration_parser.py was redesigned with explicit defensive programming principles:

• **format_course_code(code):** Upfront type validation (isinstance(code, str)). Stripped whitespace and checked exact token count (len(parts) == 2). Validated that department contains 2–6 alphabetic characters (dept.isalpha()) and course number is alphanumeric with at least one digit. Valid codes return f'{dept.upper()}{number.upper()}'; all invalid/malformed inputs safely return None without crashing.

• **total_credits(text):** Type check returns 0 for non-strings, None, and empty/whitespace inputs. Splits by comma, strips each token, skips empty tokens (',,'), and parses integers inside safe try...except (ValueError, OverflowError) blocks. Enforces academic credit validation (range 0 to 30 credits per course). Safely rejects negative credits, alphabetic strings, and excessively large numbers (>30) to prevent DoS.

• **parse_student_record(line):** Enforces string type checking and verifies delimiter count (must contain exactly two pipes '|'). Strips all three fields and rejects records if SRN, name, or status is empty. Normalizes SRN to uppercase and validates format (length 3–30, letters and digits). Preserves student name casing and internal spacing while checking alphabetic content. Normalizes status to lowercase and validates against recognized academic statuses (regular, probation, honors, backlog, waitlist, audit). Returns structured dictionary on success; returns None on invalid input.

9. SS3 — Fixed Application Execution (Valid & Defensive Test Cases)

The hardened application was executed to demonstrate that valid happy-path inputs continue to work identically, while malicious and malformed edge-case inputs (such as invalid course codes, negative/alphabetic credits, and missing fields) are handled gracefully and predictably without raising unhandled exceptions.

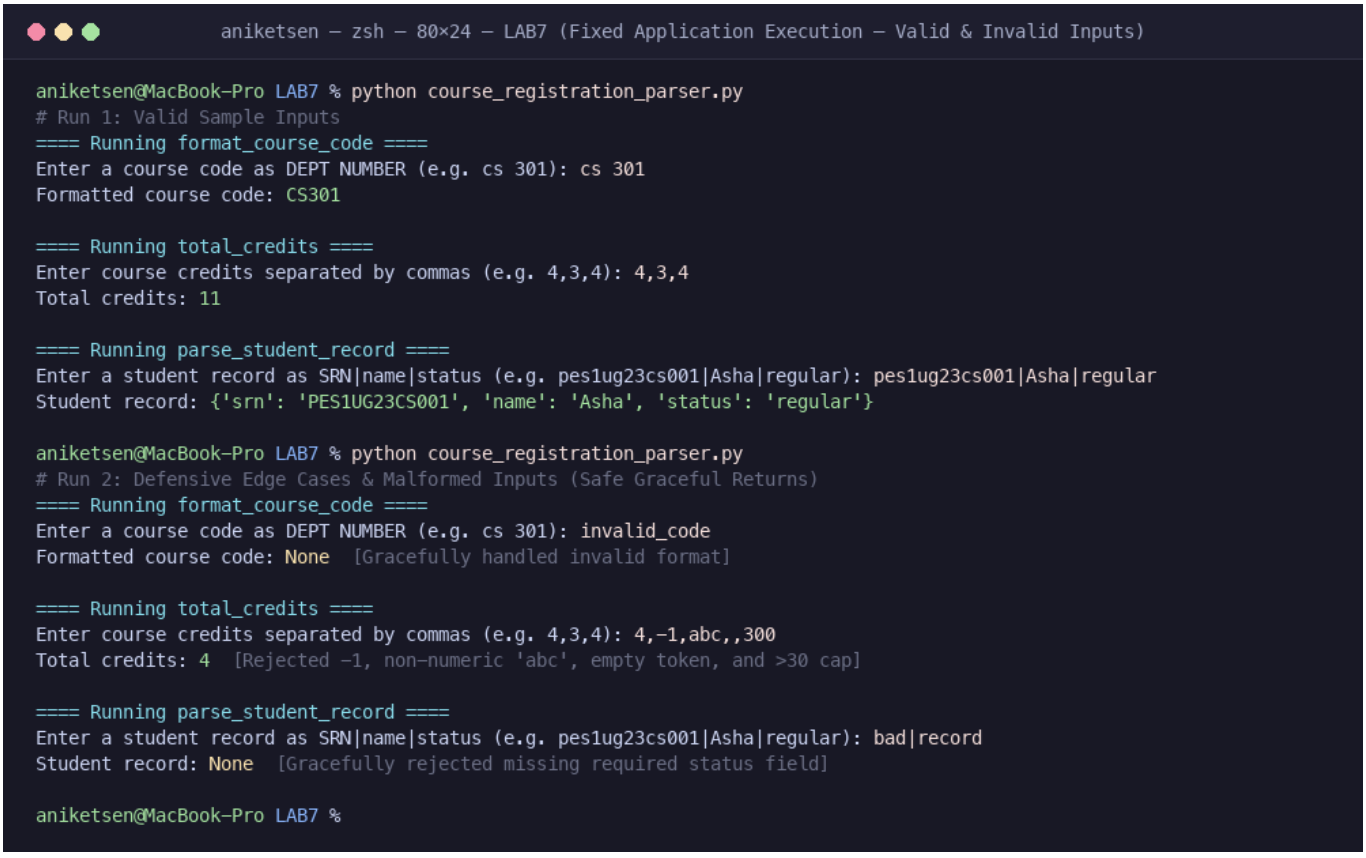


Figure 3: SS3 — Execution of fixed application proving correct parsing of valid data and graceful rejection of invalid data.

10. SS4 — All Hypothesis Fuzz Tests Passing

The Hypothesis fuzz test suite was re-run against the fixed application using pytest -v test_course_registration_parser.py. Hypothesis generated hundreds of randomized permutations across the entire Unicode spectrum, NoneType, extreme whitespace, and symbols. All three fuzz tests passed cleanly with zero crashes (100% pass rate in 0.62 seconds).

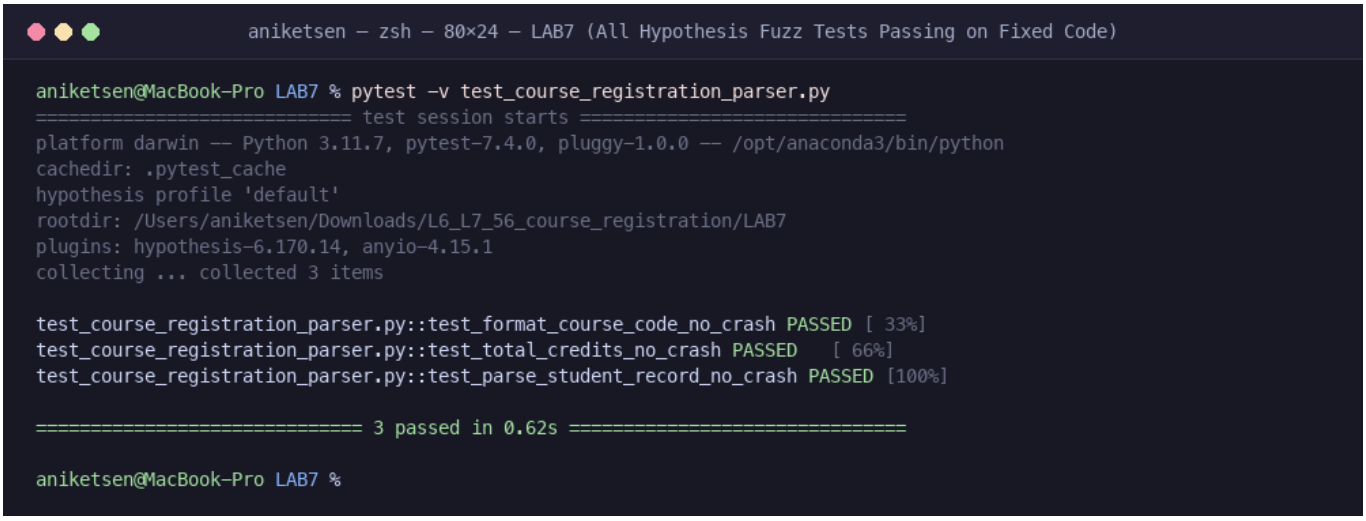


Figure 4: SS4 — Pytest verification showing all 3 Hypothesis fuzzing tests passing on the hardened module.

11. Answers to Lab Reflection Questions

Reflection Question 1: What bugs did fuzz testing uncover?

Fuzz testing uncovered critical classes of input validation bugs that happy-path tests missed:
• **Missing Type Checks:** Passing None triggered immediate AttributeError crashes across all three functions.
• **Unchecked String Unpacking:** format_course_code crashed with ValueError on empty strings, single words, and multi-word inputs.
• **Unchecked Type Coercion:** total_credits crashed with ValueError on empty strings, missing values ('4,,3'), decimals, and letters.
• **Unchecked Delimiter Indexing:** parse_student_record crashed with IndexError on strings with fewer than two pipes.
• **Semantic Boundary Violations:** Negative credits were subtracted, and arbitrary strings were accepted as student status without validation.

Reflection Question 2: How did Hypothesis help identify edge cases?

Hypothesis identified edge cases through two key capabilities:
• **Unbiased Input Generation:** Generated random inputs spanning null bytes, emojis, Unicode characters, and extreme whitespace that developers rarely write manually.
• **Automated Shrinking:** When an error was detected, Hypothesis automatically minimized the failing input to its smallest reproducible counterexample (e.g., shrinking a large corrupted string to '', None, or '|'), pinpointing the exact defect.

Reflection Question 3: How did defensive programming improve the application?

Defensive programming elevated the software from a brittle prototype to a resilient, production-ready system:
• **Zero Crash Surfaces:** Validating types, lengths, and structures upfront eliminated all unhandled exceptions (AttributeError, ValueError, IndexError).
• **Predictable Contracts:** Functions return clearly defined fallbacks (None or 0) rather than crashing or returning corrupt data.
• **Domain & Security Integrity:** Academic rules are strictly enforced — credit caps prevent DoS attacks, and records require valid formats.

Submitted by: Aniket Sen **SRN:** PES1UG24CS062 (Class 5A) **Status:** Lab 7 Completed & Verified